Lab Mini Project - Guideline & Instruction

(Complex Engineering Problem)

Department of CSE

Course Code: CSE222 Course Title: Object Oriented Programming Lab

Project Type: Mini Project (Group-based)
Total Marks: 40

Targeted Attributes for Complex Engineering Problem:

Attribute	Description
EP1	Depth of Knowledge Required
EP2	Range of Conflicting Requirements
EP3	Depth of Analysis Required
EP6	Extent of Stakeholder Involvement

Instructions to All Student Groups:

1. Group Distribution & Assigned Topics

Group No.	Project Title	Problem Domain
Group 1	Queueless Campus	EdTech
Group 2	Resort Management System	Hospitality & Tourism Management
Group 3	29 card game	Gamification
Group 4	Government-Citizen Communication Platform	E-Governance
Group 5	Smart Home Automation System	Smart Living

Each team of students (with a designated Team Lead) will tackle a **Complex Engineering Problem (CEP)** involving advanced **Object-Oriented Programming (OOP)** concepts in Java (encapsulation, inheritance, polymorphism, abstract classes, and interfaces), UML modeling, GUI, and exception handling. You must follow a structured process starting from **Requirement Analysis** to **Presentation and Documentation**.

2. Project Components & Submission Structure

The 40 marks are divided into three major assessment tools: Lab Project Report (20 Marks), Project Demonstration (10 Marks), and Followup & Viva Voce (5+5 Marks).

Part 1: Lab Project Report [20 Marks]

Your report must strictly follow the provided template and include:

- **A. Introduction (EP6) - [5 Marks]:** Define the **Library Management System**, identify diverse stakeholders (e.g., system administrators, faculty, and student borrowers), constraints, and real-world complexity.
- **B. System Architecture (EP1) - [5 Marks]:** Provide **UML Class Diagrams**, **UML Sequence & Activity Diagrams**, **Modular OOP Design Principles**, and **Java Source Code Structure** with appropriate **Access Modifiers & Encapsulation Rules**.
- **C. Implementation and Results (EP2) - [5 Marks]:** Develop comprehensive **Java Classes/Socket Programs/UML Models**, conduct exploratory analysis, test hypotheses, and resolve trade-offs between conflicting requirements (e.g., **Concurrency/Thread-Safety** vs. **System Performance/Response Time**).
- **D. Engineering Standards and Mapping (EP3) - [5 Marks]:** Evaluate the **Socio-technical and Operational** impact, ethical handling of **User Data & System Passwords**, and map the project activities to **PO, CO, and Engineering Attributes (EP1–EP6)**.

Part 2: Project Demonstration [10 Marks]

- **E. Project Demonstration (All) - [10 Marks]:** Live **execution and functional walkthrough** of the **Java-based Multi-threaded Library Management System**, demonstrating **Real-time Concurrent Borrowing**, **Client-Server Socket Communication & Search**, and **Exception Recovery under Invalid Inputs**.

Part 3: Followup & Viva Voce [5+5 Marks]

- **F. Viva Voce (All) - [5 Marks]:** Oral defense of the project, explaining **OOP Design Pattern Choices & Synchronization Logic**, **UML Class & Sequence Architecture**, and understanding of applied **Object-Oriented Software Engineering** principles.
- **G. Project Followup - [5 Marks]:** Demonstrate completion progress and Level of Understanding.

3. Deliverables & Timeline:

Deliverables:

You must submit the following:

- ✔ Github Repository with Customized README.md file
- ✔ Project Report
- ✔ Presentation

Timeline:

- **Report Submission Deadline:** 15.08.2026
- **Presentation & Viva (Optional):** 15.08.2026

4. Evaluation Rubric (Total 40 Marks)

Criteria	Description	Attribute	Deliverables	Marks
A. Introduction	Clear problem definition, project objectives, background, scope, and relevance.	EP1, EP6	Lab Project Report	5
B. System Architecture	Comprehensive system design and UML modeling (class, sequence, and activity diagrams).	EP3	Lab Project Report	5
C. Implementation and Results	Functional Java implementation using OOP principles, multi-threading, networking, and results analysis.	EP1, EP2	Lab Project Report	5
D. Engineering Standards and Mapping	Addressing engineering standards, trade-offs, ethics, sustainability, and CO-PO alignment.	EP2, EP3	Lab Project Report	5
E. Project Demonstration	Live working execution of software, user interface design, and functional feature evaluation.	EP1, EP2	Project Demonstration	10
F. Followup & Viva Voce	Individual technical defense, code understanding, and response to evaluator questions.	EP1, EP3	Viva Voce	10
			Total	**40**

Appendix

How can we address [e.g., EP1, EP2, EP3 and EP4] through the Object Oriented Programming Mini Project?

EP1 - Depth of Knowledge Required

Goal: Apply your theoretical knowledge from **advanced Object-Oriented Programming (OOP) concepts and software design paradigms** in a practical, real-world project.

How to address:

- Use your understanding of **UML class diagrams and structural modeling** to design the **Library Management System**.
- Apply **OOP principles (encapsulation, inheritance, abstraction, and polymorphism)** to **achieve code modularity and remove redundant code**.
- Develop/Write **object-oriented Java applications** using **IDE tools (Eclipse/IntelliJ), multi-threading, and socket networking** to solve real tasks like **user authentication, real-time transaction processing, or socket communication**.
- Implement **robust exception handling (try-catch blocks and custom exceptions)** to ensure **system reliability and data integrity**.

Example: Create a **UML Class Diagram** for a **multi-user Library Management System** that ensures all **user roles, transactions, and book inventories** are logically structured and interrelated.

EP2 - Range of Conflicting Requirements

Goal: Apply your theoretical knowledge from **concurrent execution and client-server architecture** in a practical, real-world project.

How to address:

- Use your understanding of **multi-threading and synchronization** to design the **concurrent transaction processing module**.

- Apply **thread-safety mechanisms and resource locking** to **prevent race conditions and avoid data corruption during simultaneous user updates**.
- Develop/Write **network socket handlers** using **Java Socket API** to solve real tasks like **handling remote client connections, real-time inventory synchronization, or managing concurrent borrow requests**.
- Implement **input validation and exception handling** to ensure **system stability under high user load**.

Example: Create a **thread synchronization mechanism** for a **multi-user borrowing environment** that ensures all **book availability statuses, transaction logs, and user balances** are logically structured and interrelated without data conflicts.

EP3 - Depth of Analysis Required

Goal: Apply your theoretical knowledge from **system analysis and design alternative evaluations** in a practical, real-world project.

How to address:

- Use your understanding of **UML sequence and activity diagrams** to design the **system workflows and interaction models**.
- Apply **architectural trade-off analysis** to **balance system performance against implementation complexity**.
- Develop/Write **algorithmic solutions for data management** using **appropriate collections and data structures** to solve real tasks like **searching book records, sorting user accounts, or tracking active socket sessions**.
- Implement **performance profiling and unit test cases** to ensure **optimal execution efficiency and system accuracy**.

Example: Create a **Sequence Diagram** for a **remote transaction flow** that ensures all **client requests, server validation steps, and database response loops** are logically structured and interrelated.

EP6 - Extent of Stakeholder Involvement

Goal: Apply your theoretical knowledge from **stakeholder requirement analysis and user-centric design** in a practical, real-world project.

How to address:

- Use your understanding of **use-case analysis and user role mapping** to design the **interface and permission levels**.
- Apply **role-based access control guidelines** to **deliver distinct views for administrators versus regular users**.
- Develop/Write **user interface components and feedback loops** using **Java UI frameworks or interactive console interfaces** to solve real tasks like **displaying error notifications, reporting transaction statuses, or presenting system activity summaries**.
- Implement **simulated stakeholder feedback integration** to ensure **usability, accessibility, and functional compliance**.

Example: Create an **Interface Hierarchy** for a **role-based system** that ensures all **admin functions, regular user permissions, and guest actions** are logically structured and interrelated.